

Guia de Execução do Sistema

Recomendação de Refatorações via Debate Multiagente — passo a passo para testar e avaliar a ferramenta

Autor: Gabriel Al-Samir G. Sales · **Orientador:** Marcos Antonio de Oliveira **Instituição:** Universidade Federal do Ceará (UFC) — Campus Quixadá

Este guia explica, do zero, como instalar e executar o sistema implementado a partir do artigo *"Recomendação de Refatorações via Debate Multiagente: Uma Arquitetura Inspirada em Revisão por Pares"*. O objetivo é permitir que o avaliador rode a ferramenta localmente, observe o debate entre os agentes especialistas e o Agente Juiz, e confira que a implementação corresponde à arquitetura descrita no artigo (Seção 4, Figura 1).

Não é necessário conhecimento prévio do código — apenas um terminal.

1. Visão geral do que você vai executar

O sistema recebe um arquivo Python e produz uma **recomendação consolidada de refatorações**, construída por quatro agentes:

- 1. **Agente de Sustentabilidade de Software** — impacto energético e desperdício de recursos;
- 2. **Agente de Arquitetura** — coesão, acoplamento e boas práticas;
- 3. **Agente de Desempenho** — gargalos computacionais e hot paths;
- 4. **Agente de Debate (Juiz)** — confronta as três visões, expõe conflitos de design e consolida a recomendação final, negociando os *trade-offs*.

Essas análises são fundamentadas em métricas determinísticas extraídas da Árvore de Sintaxe Abstrata (AST) e de ferramentas externas (Radon, Pylint, import-linter, SonarQube, Scalene, py-spy, cProfile, CodeCarbon) — exatamente a arquitetura da Figura 1 do artigo.

Você poderá rodar o sistema de duas formas: pela **linha de comando** (mais rápido para avaliar) ou pela **API REST** (como descrito no artigo, via FastAPI).

2. Pré-requisitos

Requisito	Obrigatório?	Observação
macOS, Linux ou Windows (WSL)	sim	testado em macOS
Git	sim	para obter o código
uv (gerenciador Python)	sim	instala o Python 3.12 automaticamente
Ollama (modelos locais)	não	sem ele, o sistema usa um modo determinístico equivalente

Docker	não	apenas para o SonarQube opcional
--------	-----	----------------------------------

2.1. Instalar o uv

O uv gerencia o Python e as dependências do projeto — não é necessário instalar Python manualmente.

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

(Windows: ver instruções em <https://docs.astral.sh/uv/getting-started/installation/>.)

3. Obtendo o projeto

```
git clone <URL-do-repositorio> ai-multi-agent-refactoring-debate
cd ai-multi-agent-refactoring-debate
```

Se você recebeu o projeto como uma pasta/zip em vez de um link do Git, basta extrair e abrir um terminal dentro da pasta ai-multi-agent-refactoring-debate.

4. Instalação

Um único comando instala o Python 3.12 (isolado, não afeta o sistema) e todas as dependências (FastAPI, CrewAI, Radon, Pylint, Scalene, CodeCarbon, etc.):

```
uv sync --extra dev
```

Em seguida, copie o arquivo de configuração padrão:

```
cp .env.example .env
```

Não é necessário editar o `.env` para o primeiro teste — os valores padrão já funcionam.

5. Primeiro teste — rodando pela linha de comando

O repositório já inclui um arquivo de exemplo com problemas propositais em todas as três dimensões (examples/sample_code.py). Rode:

```
uv run refactoring-debate examples/sample_code.py
```

O que esperar na tela

A ferramenta exibe, em sequência:

1. **Cabeçalho** — nome do arquivo, modelo de LLM usado e tempo total.

2. **Relatório de cada especialista** — as recomendações que cada agente propôs, isoladamente, dentro da sua área.
3. **Conflitos de design (Q3)** — onde dois agentes discordam (ex.: Desempenho quer *cache*, Sustentabilidade aponta que isso aumenta o consumo de energia).
4. **Trade-offs negociados** — como o Agente Juiz resolveu o conflito, e por quê.
5. **Tabela consolidada** — a lista final de recomendações, com prioridade, status (aceito / mesclado / postergado) e severidade.
6. **Indicadores de validação** — quantos atributos de qualidade distintos foram cobertos (Q1/Q2) e quantos conflitos emergiram (Q3), conforme o plano de validação do artigo (Seção 4.3).

Isso comprova, na prática, o fluxo da Figura 1 do artigo: AST → ferramentas → agentes especializados → debate → saída consolidada.

Testando com análise dinâmica (opcional)

Para também medir CPU/memória/energia reais (Scalene, cProfile, CodeCarbon), execute:

```
uv run refactoring-debate examples/sample_code.py --dynamic
```

Esse modo **executa o código analisado**. Use apenas com arquivos confiáveis — por padrão, fica desativado.

Testando com um arquivo próprio

```
uv run refactoring-debate caminho/para/seu_arquivo.py
```

Ou cole código diretamente:

```
echo "def f(x):  
    return x" | uv run refactoring-debate -
```

6. Segundo teste — rodando a API REST (como descrito no artigo)

O artigo propõe a exposição do sistema via API REST em FastAPI. Para testar:

```
uv run uvicorn refactoring_debate.main:app --reload
```

Abra o navegador em:

```
http://localhost:8000/docs
```

Essa é a interface interativa (Swagger UI) gerada automaticamente pelo FastAPI. Nela, você pode:

- expandir GET `/health` → clicar em **Try it out** → **Execute**, para confirmar que o serviço está no ar e ver quais ferramentas estão disponíveis;
- expandir POST `/api/v1/analyze` → **Try it out** → colar um código Python no campo `code` → **Execute**, para obter a recomendação consolidada em JSON, incluindo o registro completo do debate (`debate.conflicts`, `debate.tradeoffs`).

Testando via terminal (alternativa ao navegador)

Com o servidor rodando (passo anterior, em outra aba do terminal):

```
curl -s http://localhost:8000/api/v1/analyze \
-H "content-type: application/json" \
-d '{"filename": "exemplo.py", "code": "def f(x):\n    out = []\n    for i in range(len(x)):\n        for
```

A resposta JSON traz os relatórios de cada agente, os conflitos detectados, os trade-offs negociados e a lista consolidada de recomendações.

Para encerrar o servidor, volte ao terminal onde ele está rodando e pressione `Ctrl+C`.

7. Como interpretar o que o sistema produz

Seção da saída	O que significa	Onde está no artigo
Relatório de cada agente	Recomendações locais, vistas apenas pela métrica daquele especialista	Seção 4.2, "Agentes Especializados"
Conflitos de design	Tensão explícita entre dois agentes sobre o mesmo trecho de código	Seção 4.1, Debate Multiagente (MAD)
Trade-offs	Como o Juiz decidiu, usando pesos de prioridade configuráveis	Seção 4.2, "Agente de Debate"
Tabela consolidada	Saída final, priorizada, do sistema	Figura 1, etapa 6
Indicadores Q1/Q2/Q3	Diversidade de recomendações, amplitude de atributos cobertos, conflitos emergentes	Seção 4.3, Plano de Validação

8. Rodando os testes automatizados (opcional, mas recomendado)

O projeto tem uma suíte de testes automatizados que comprova que cada peça do sistema funciona corretamente — útil para a avaliação técnica.

```
uv run pytest
```

Resultado esperado: todos os testes passam (formato `26 passed`). Para conferir também a qualidade do código:

```
uv run ruff check .      # estilo e boas práticas
uv run mypy src          # verificação de tipos
uv run lint-imports      # contratos arquiteturais do próprio projeto
```

9. Usando um modelo de linguagem real (opcional)

Por padrão, o sistema funciona **sem nenhum modelo de linguagem externo**: os agentes usam regras determinísticas fundamentadas nas métricas (modo `heuristic`), o que já é suficiente para observar todo o fluxo de debate descrito no artigo.

Para usar o backend proposto no artigo (Ollama + Llama3, modelo local e gratuito):

```
# instalar o Ollama (uma vez)
brew install ollama           # macOS – outras plataformas: https://ollama.com/download
ollama serve &
ollama pull llama3

# no arquivo .env, garantir:
#   RD_LLM_PROVIDER=ollama
#   RD_LLM_MODEL=ollama/llama3
```

A partir daí, basta repetir o comando da Seção 5 — agora os agentes raciocinam com o modelo de linguagem em vez das regras determinísticas.

Se o Ollama não estiver disponível no momento da execução, o sistema **detecta isso automaticamente** e usa o modo determinístico, sem travar — uma decisão de robustez da implementação.

10. Solução de problemas

Sintoma	Causa provável	Solução
uv: command not found	uv não instalado ou terminal não recarregado	Reabra o terminal após instalar o uv
Erro ao instalar dependências	Versão de Python incompatível no sistema	Não é necessário Python prévio: o uv baixa o 3.12 sozinho
LLM: heuristic mesmo configurando Ollama	Ollama não está rodando ou modelo não baixado	Confirme com <code>ollama serve</code> e <code>ollama pull llama3</code>
Página do Swagger não abre	Servidor não está rodando	Confira se o comando da Seção 6 ainda está ativo no terminal
Quero parar o servidor	—	<code>Ctrl+C</code> no terminal onde o uvicorn está rodando

11. Checklist rápido para a avaliação

```
# 1. instalar
uv sync --extra dev
cp .env.example .env

# 2. rodar o exemplo pela linha de comando
uv run refactoring-debate examples/sample_code.py
```

```
# 3. rodar a API e abrir o Swagger
uv run uvicorn refactoring_debate.main:app --reload
#   -> http://localhost:8000/docs

# 4. rodar os testes automatizados
uv run pytest
```

Com esses quatro passos, é possível observar a arquitetura completa do artigo em funcionamento: extração de métricas via AST, debate entre os três agentes especialistas, arbitragem do Agente Juiz e exposição via API REST.

Dúvidas sobre este guia ou sobre a implementação podem ser direcionadas ao autor.